

Topic	Decision / Details
Object-oriented programming language (if not using OOP, then justify the use of the language)	Python 3.12 (object-oriented; supports classes, inheritance, method overriding, polymorphism, and dynamic binding)
GUI library (strongly encouraged)	Tkinter (Python's standard GUI library; built into the Python standard library, no extra install needed; cross-platform)
IDE (Integrated Development Environment)	Visual Studio Code with the Microsoft Python extension
xUnit framework (e.g., JUnit for Java)	pytest (with parametrize for data-driven tests)
Programming style guide (must read it carefully)	PEP 8 (https://peps.python.org/pep-0008/) enforced with the Black code formatter
Project hosting site	https://github.com/MChaudhary123/Intro-to-Software-Engineer
Other decisions if applicable	Version control: Git + GitHub; virtual environment via <code>python -m venv</code> ; dependencies pinned in requirements.txt

Part 2 - Unit Testing

Made a quick calculator and tested it.

Source Code:

calculator.py

```
"""A simple calculator module for unit testing practice."""
```

```
def add(a, b):
```

```
    """Return the sum of a and b."""
```

```
    return a + b
```

```
def subtract(a, b):  
    """Return a minus b."""  
    return a - b
```

```
def multiply(a, b):  
    """Return a times b."""  
    return a * b
```

```
def divide(a, b):  
    """Return a divided by b. Raises ValueError if b is zero."""  
    if b == 0:  
        raise ValueError("Cannot divide by zero.")  
    return a / b
```

test_calculator.py

```
"""Unit tests for calculator.py using pytest."""
```

```
import pytest  
from calculator import add, subtract, multiply, divide
```

```
def test_add():  
    assert add(2, 3) == 5  
    assert add(-1, 1) == 0
```

```
def test_subtract():  
    assert subtract(10, 4) == 6  
    assert subtract(0, 5) == -5
```

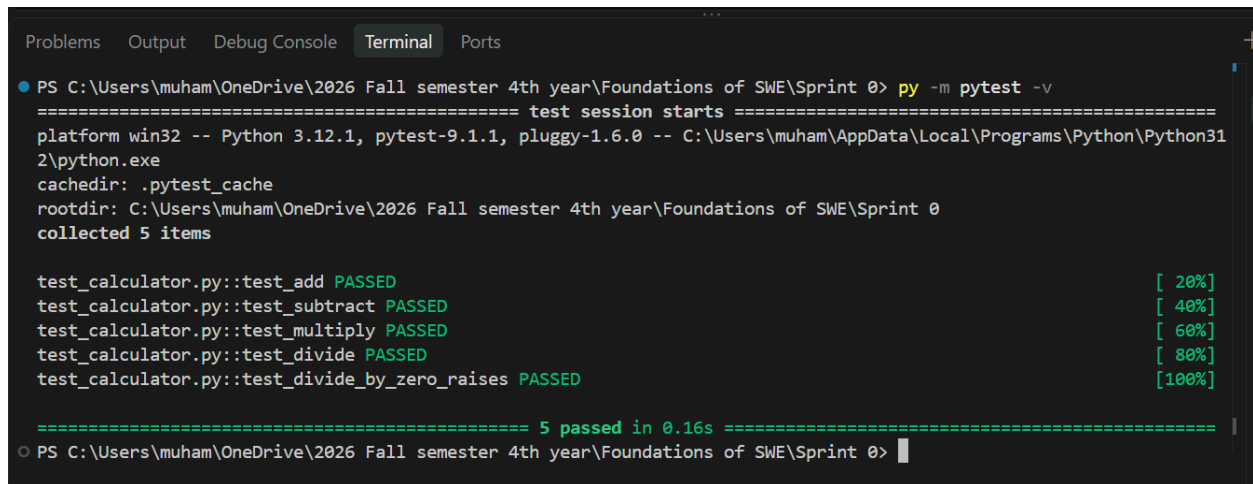
```
def test_multiply():  
    assert multiply(3, 4) == 12
```

```
assert multiply(-2, 5) == -10
```

```
def test_divide():  
    assert divide(10, 2) == 5  
    assert divide(9, 3) == 3
```

```
def test_divide_by_zero_raises():  
    with pytest.raises(ValueError):  
        divide(5, 0)
```

Output:



```
PS C:\Users\muham\OneDrive\2026 Fall semester 4th year\Foundations of SWE\Sprint 0> py -m pytest -v  
===== test session starts =====  
platform win32 -- Python 3.12.1, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\muham\AppData\Local\Programs\Python\Python312\python.exe  
cachedir: .pytest_cache  
rootdir: C:\Users\muham\OneDrive\2026 Fall semester 4th year\Foundations of SWE\Sprint 0  
collected 5 items  
  
test_calculator.py::test_add PASSED [ 20%]  
test_calculator.py::test_subtract PASSED [ 40%]  
test_calculator.py::test_multiply PASSED [ 60%]  
test_calculator.py::test_divide PASSED [ 80%]  
test_calculator.py::test_divide_by_zero_raises PASSED [100%]  
  
===== 5 passed in 0.16s =====  
PS C:\Users\muham\OneDrive\2026 Fall semester 4th year\Foundations of SWE\Sprint 0>
```

Part 3 – GUI Programming

Gui_demo.py

```
import tkinter as tk
```

```
def main():  
    window = tk.Tk()  
    window.title("Sprint 0 GUI Demo")  
    window.geometry("400x350")
```

```

# --- TEXT ---
title = tk.Label(window, text="Sprint 0 GUI Demo",
                  font=("Arial", 16, "bold"))
title.pack(pady=10)

subtitle = tk.Label(window, text="By Muhammad - CS 449",
                    font=("Arial", 10))
subtitle.pack()

# --- LINES ---
# A horizontal line drawn on a Canvas.
canvas = tk.Canvas(window, width=350, height=40, highlightthickness=0)
canvas.pack(pady=10)
canvas.create_line(10, 20, 340, 20, fill="black", width=2)
canvas.create_line(10, 30, 340, 30, fill="gray", width=1)

# --- RADIO BUTTONS ---
tk.Label(window, text="Pick a color:",
          font=("Arial", 11, "bold")).pack()

color_choice = tk.StringVar(value="Red")
for color in ["Red", "Green", "Blue"]:
    tk.Radiobutton(window, text=color, variable=color_choice,
                   value=color).pack(anchor="w", padx=100)

# --- CHECK BOX ---
agree = tk.BooleanVar()
tk.Checkbutton(window, text="I agree to the terms",
               variable=agree).pack(pady=10)

# --- BUTTON that shows what was selected ---
result = tk.Label(window, text="", font=("Arial", 10), fg="blue")
result.pack()

def show_selection():
    result.config(
        text=f"Color: {color_choice.get()} | Agreed: {agree.get()}"
    )

tk.Button(window, text="Show Selection",

```

```
command=show_selection).pack(pady=5)
```

```
window.mainloop()
```

```
if __name__ == "__main__":  
    main()
```

Output:

